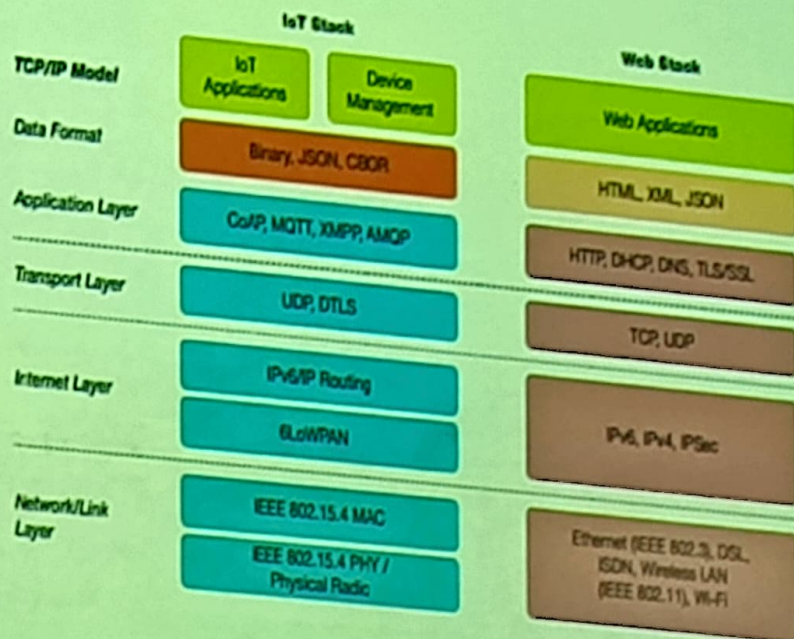


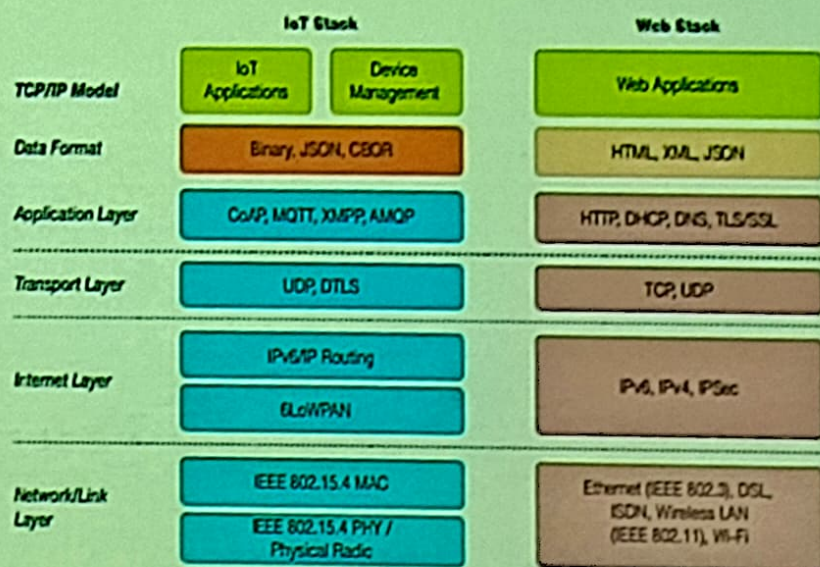
Stack in Brief

- Application Layer — HTTP, MQTT, gRPC
- Transport Layer — TCP/UDP
- Internet Layer — IP
- Link Layer — Ethernet/Wi-Fi



Network Stack in Brief

- Application Layer — HTTP, MQTT, gRPC
- Transport Layer — TCP/UDP
- Internet Layer — IP
- Link Layer — Ethernet/Wi-Fi



- Two entities: client (requests) and server (responds).
- Follows the request-response paradigm.
- Example: Browser requesting data from a web API.

A Typical HTTP Request



```
GET /api/data HTTP/1.1  
Host: example.com  
Accept: application/json
```

```
HTTP/1.1 200 OK  
{  
  "temperature": 24.7,  
  "unit": "C"  
}
```

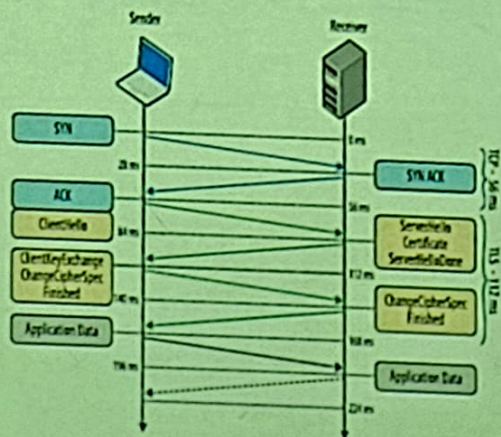

Transport Layer Security (TLS) Overview

- TLS ensures **Confidentiality, Integrity, and Authentication** in client-server communication.
- It operates on top of TCP, securing protocols like **HTTPS, FTPS, IMAPS**.
- Core components:
 - **Encryption**: Prevents eavesdropping using symmetric keys.
 - **Authentication**: Server proves its identity using X.509 certificates.
 - **Integrity**: Message Authentication Codes (MACs) detect tampering.
- TLS uses a combination of **asymmetric** (RSA, ECDSA) and **symmetric** (AES, ChaCha20) cryptography.

TLS Handshake Process

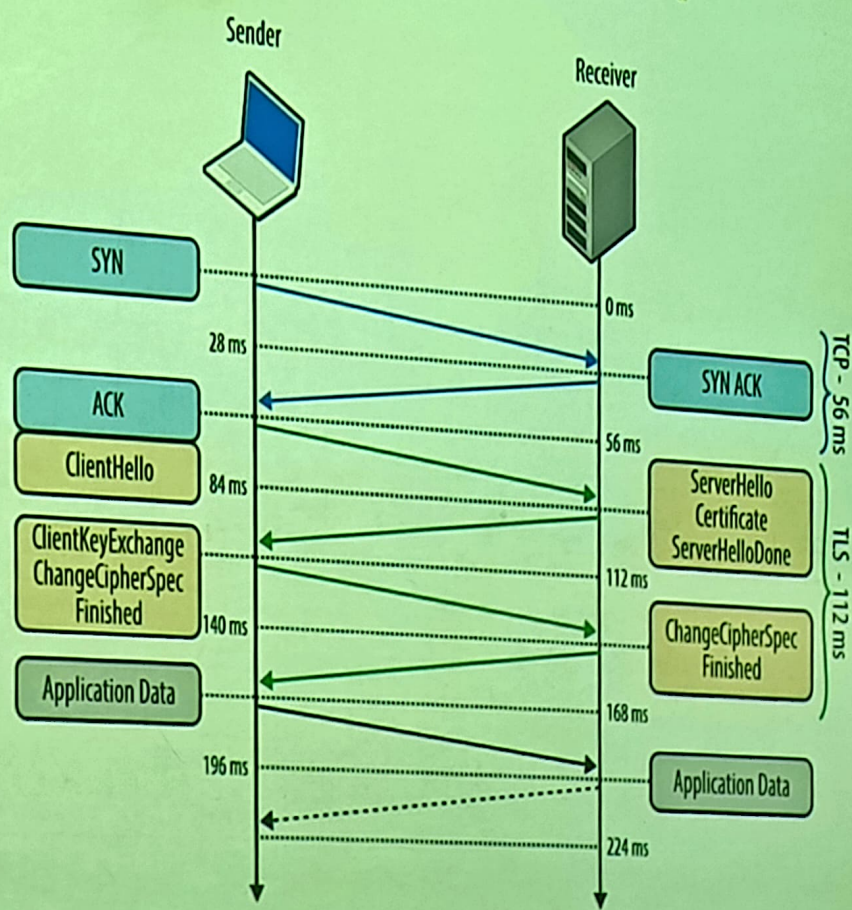
- 1 **ClientHello:** Client sends supported TLS versions, cipher suites, and a random nonce.
- 2 **ServerHello:** Server selects parameters, sends its certificate, and its own random nonce.
- 3 **Key Exchange:** Using RSA, Diffie-Hellman, or ECDHE, both derive a shared secret.

- 1 **Session Keys:** From the shared secret, symmetric session keys are generated.
- 2 **Finished Messages:** Both sides verify handshake integrity.
- 3 **Encrypted Communication Begins.**



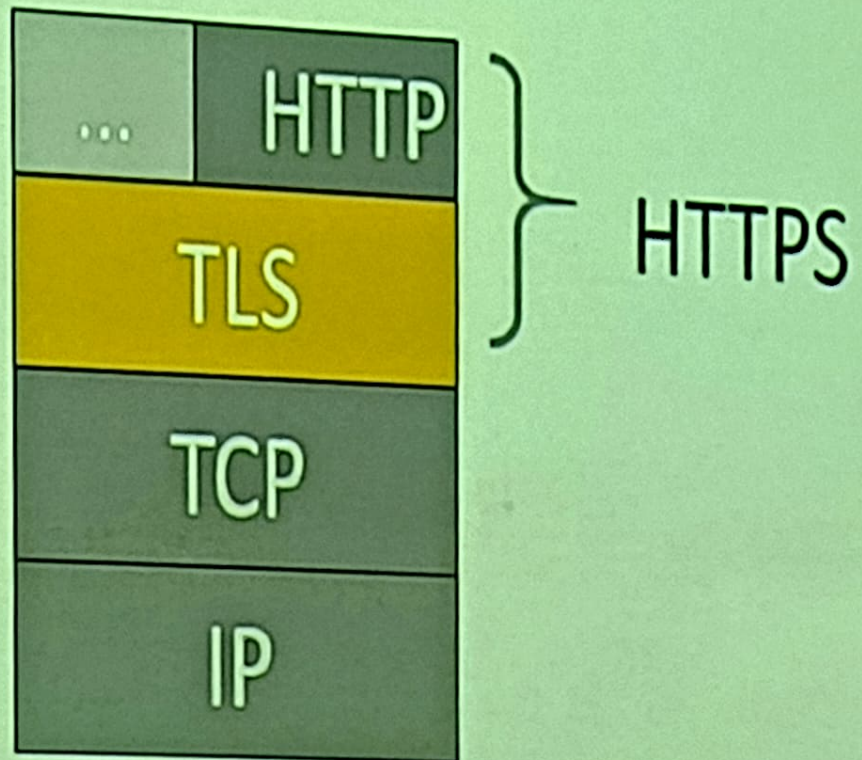
secret.

Con
Beg



HTTP vs HTTPS

- HTTPS adds encryption via TLS.
- Ensures authentication, integrity, and confidentiality.
- Widely used for web and mobile applications.



Key Properties

Advantages

- Simple and universal.
- Easy to debug and test.
- Works across platforms.

Limitations

- Stateless.
- Overhead with headers.
- Not optimal for streaming.

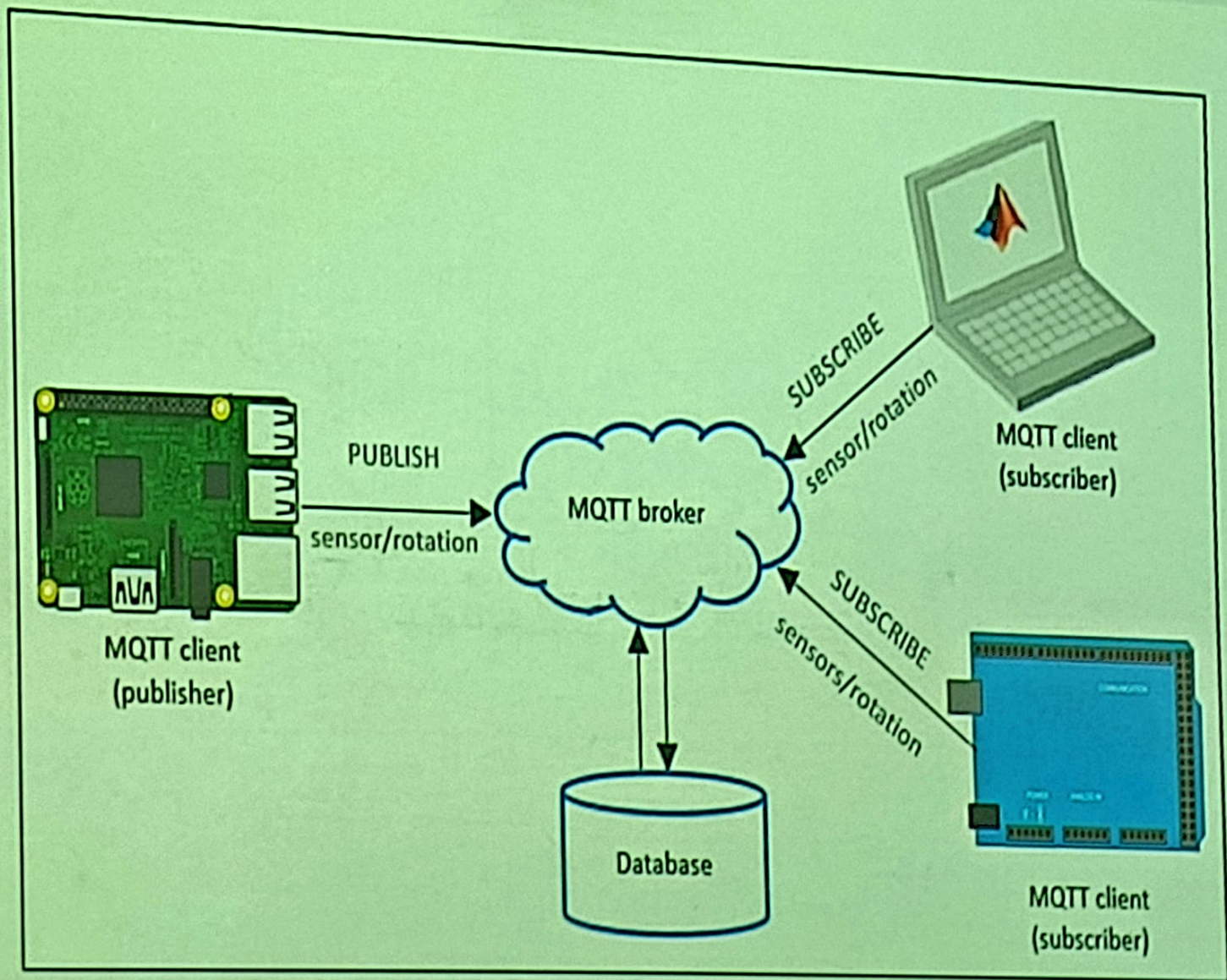
Motivation for Pub-Sub

- Event-driven systems and IoT devices need lightweight communication.
- Direct request-response is inefficient for many small devices.
- Enter the publish-subscribe model.

MQTT Overview

- MQTT = Message Queuing Telemetry Transport.
- Designed for low-bandwidth and unreliable networks.
- Central Broker manages all messages.

MQTT Architecture



Demo: MQTT Commands

install clients

```
sudo apt install mosquitto mosquitto-clients
```

in one terminal

```
mosquitto_sub -t "demo/topic"
```

in another terminal

```
mosquitto_pub -t "demo/topic" -m "Hello-MQTT!"
```

Advantages of MQTT

- Asynchronous, scalable.
- Supports QoS levels (0, 1, 2).
- Very small overhead.
- Ideal for IoT sensors, smart devices, telemetry.

Drawbacks of MQTT

- Requires broker management.
- Not suitable for heavy payloads.
- Less standard for web clients.

The RPC Idea

- Instead of sending messages, call a remote function as if local.
- Hides networking under interface definition.
- Enables strongly typed communication.

The RPC Idea

- Instead of sending messages, call a remote function as if local.
- Hides networking under interface definition.
- Enables strongly typed communication.

What is gRPC?

- Developed by Google, built on HTTP/2.
- Uses Protocol Buffers for serialization.
- Supports streaming and bidirectional calls.

Service Definition

```
syntax = "proto3";
```

```
service Greeter {  
    rpc SayHello (HelloRequest) returns (HelloReply);  
}
```

```
message HelloRequest { string name = 1; }  
message HelloReply   { string message = 1; }
```

Client-Server Example (Python)

Client

```
channel = grpc.insecure_channel('localhost:50051')
stub = GreeterStub(channel)
response = stub.SayHello(HelloRequest(name="Alice"))
print(response.message)
```


gRPC Advantages

- Compact and fast binary serialization.
- Supports multiple languages.
- Full-duplex streaming.
- Great for microservice communication.

Microservices

- **Microservices Architecture** decomposes an application into small, independent services.
- Each service:
 - Performs a specific business function.
 - Runs in its own process (often in a container).
 - Communicates via lightweight protocols — typically **HTTP/REST**, **gRPC**, or **Message Queues**.
- Enables:
 - Independent deployment and scaling.
 - Fault isolation and better maintainability.
 - Technology heterogeneity — each service can use a different stack.

Microservices vs Monolithic Architecture

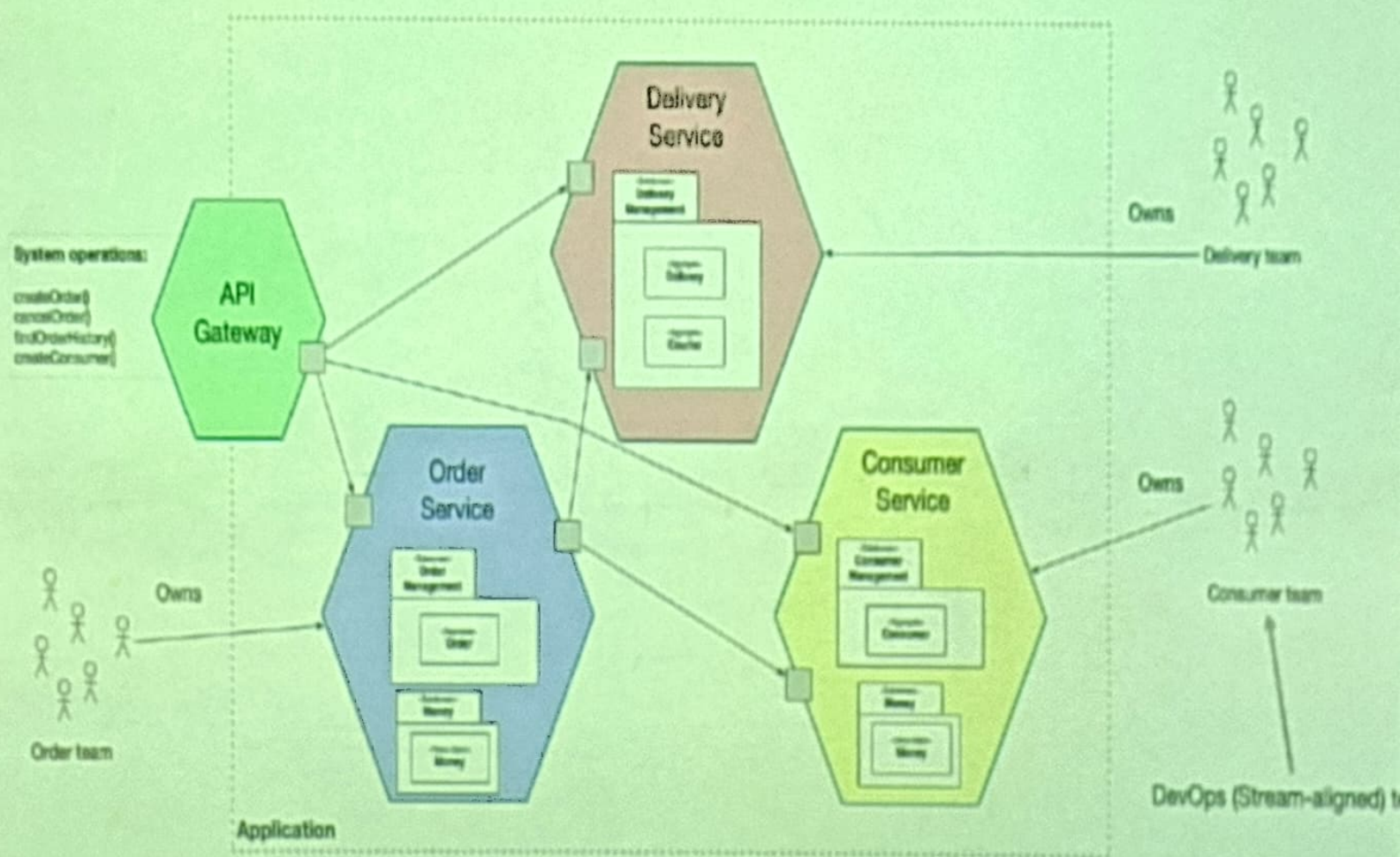
Monolithic

- Single, unified codebase.
- Tight coupling between components.
- Difficult to scale or update individual features.
- One failure can bring down the entire system.

Microservices

- Loosely coupled, independently deployable services.
- Easier continuous delivery and scalability.
- Uses APIs for inter-service communication.
- Ideal for cloud-native and containerized environments.

Microservice Example



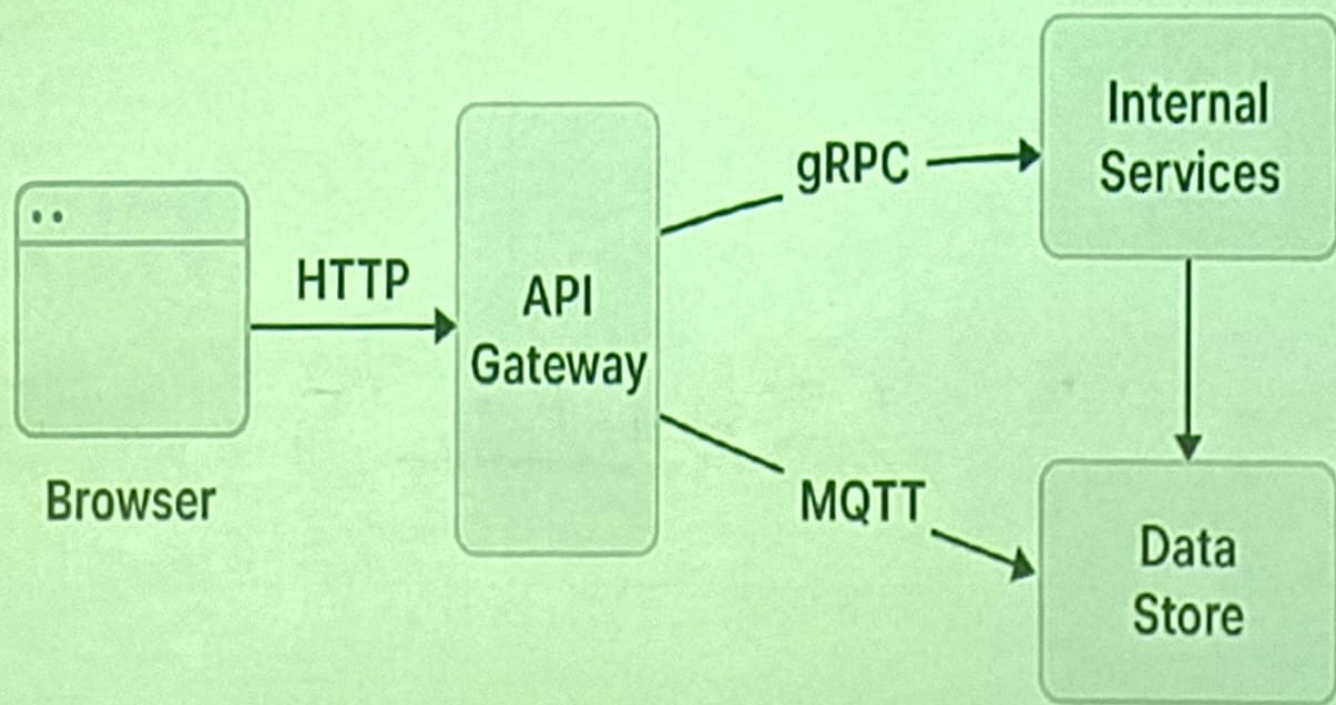
Comparing Communication Paradigms

Feature	HTTP	MQTT	gRPC
Model	Req-Resp	Pub-Sub	RPC
Transport	TCP (HTTP/1.1)	TCP	HTTP/2
Data	JSON	Binary payload	Protobuf
Stateful?	No	Optional	Yes
Use case	Web Apps	IoT	Microservices

When to Choose What

- **HTTP/HTTPS:** Simplicity, browser-friendly.
- **MQTT:** IoT, low bandwidth, event-based.
- **gRPC:** High performance backend communication.

Combined Architecture Example



Real-World Example

- IoT device sends telemetry via MQTT.
- Cloud service aggregates and exposes REST API.
- Internal services communicate via gRPC.

Security Considerations

- TLS everywhere (HTTPS, MQTTS, gRPC over TLS).
- Authentication: tokens, certificates.
- Rate limiting and DoS prevention.

Performance Insights

- Latency comparison.
- Compression benefits of Protobuf.
- Tradeoff between human readability and efficiency.

Future Trends

- GraphQL and WebSockets.
- Edge computing with MQTT bridges.
- gRPC-Web and service mesh integration.

24-Oct-25

→ Build multi person chat & Multiplayer game

→ layers in network stack, Transport, Application
IP, Data link layer,
etc.

- webpages are essentially in JSON

- where Does DNS happens in this stack?

what is ICMP? , where Does ping happen?

Can we Do Secure DNS? what it means to do secure DNS

Where Does VPN work? in which layer?

How to break into VPN? , If you don't know how

to break VPN , you won't know how to secure
VPN?

What is API business? what does it actually do

as supposed to HTTP server?

web API vs Web Server?

JDBC? → Database connectivity, is it API

When will HTTPS come? How its different from HTTP?

↳ on E-com we need security, so they
built security in browser and defined
HTTPS

TLS - Transport layer Security

↳ SSL vs TLS?

TLS is just evolution of SSL

SSL 2.0 is TLS 1.0

↳ Additional state in State transit
Diagram

Nowadays SSL and TLS are used interchangeably

TCP-Handshake?

(HTTPS uses TLS)

TLS Handshake

How do you talk to server with a Commandline?

How do you Start Communication?

Protocol used?

→ telnet, IP address and port

→ blocking things used by server to create child

- Encrypt Data, need ~~cert~~ certificate to come through, then decide protocol, encryption to use

→ use ~~Asympt~~ ^{Asymmetric} key cryptography

then for session you say its my symmetric key for RSA

Why we use symmetric?

How to debug TLS? look into some library?

→ Have only Binary file, No debugger how do you tell its iterative or Recursive version of fibo?

- Any Real machine has stack depth/memory that you have
- OS - 4GB - 1GB (OS) 3GB (user)

Given sufficiently large input, it should give stackoverflow for recursive but not for iterative

What do you mean by stateless in N/w?

Cookie approach?

↳ website ask for cookies

(Store session info)

↓
Its like browser Remember your
pswd.

MQTT Protocol??

↳ (meant to give High BW, Low latency
but works opposite)

MQTT Client, MQTT Broker, DB, (slide)

↑
Publisher / subscriber traffic routed
via broker

MQTT Commands

MQTT is not about internet, its about connectivity

↳ Hard to detect

Marshalling / Unmarshalling

video
Server

HTTPS

handling
connection

main C {
get data()
remote analytics C

RPCs
(Remote call
Procedures)

UNIX

M/C

Want to do
analytics on
another machine

Not possible to do
Distributed computing
without it

rem. Analytics()

• Must take care of endianness
Assume all communication happen
on little endianness